

Framework de Desarrollo de Software con IA (V2)

Este framework optimiza el ciclo de vida del desarrollo utilizando agentes autónomos y modelos de razonamiento avanzado, enfocándose en la persistencia del contexto y la validación continua mediante Desarrollo Guiado por pruebas.

Pre-Requisitos

Este documento está orientado al desarrollo guiado por IA, específicamente utilizando la herramienta Claude Code, ya sea en la terminal o en la aplicación de Claude.

Requerimientos

- El usuario es necesario que cuente con una cuenta de **Claude**. Otras herramientas como Open Code, Github Copilot o Antigravity puede que funcione, pero esta guía está probada y optimizada para Claude Code
- El usuario debe dominar el uso de los Skills, para saber que son los skills consultar: <https://platform.claude.com/docs/en/agents-and-tools/agent-skills/overview>

Detalle de Fases y Argumentación Técnica

1. Extracción de Requisitos Funcionales y optimización de los mismos para modelos de lenguaje.

El proceso de desarrollo actual, que se basa entre otras cosas en procesos de desarrollo ágiles como SCRUM, tiene unas bases sólidas que pueden ayudar en el proceso de desarrollo guiado por AI. Pero para esto, cada miembro del equipo debe adaptarse a esta nueva normalidad.

Uno de los artefactos que genera el proceso de desarrollo con SCRUM, son los “*Documentos de Requisitos Funcionales*”. Estos documentos muchas veces están basados en plantillas ya previamente trabajadas que son sumamente **verbosas** y están **hechas** para que sean entregadas a clientes y para que otros “*HUMANOS*”, la puedan leer y entender. En el caso de los modelos de lenguajes (GPT, CLAUDE, etc.), muchas veces estos documentos tienen muchísima información que no les es para nada relevante, además de adicionar muchísimos **tokens** a la ventana de contextos.

Es por eso que surgen los formatos de documentos que están optimizados para AI, el

objetivo de este documento es ser lo suficientemente claro y **conciso** para evitar el uso excesivo de **tokens** y las formalidades.

Anteriormente existía un documento muy usado llamado Product Requirement Document (PRD). Este documento describe las funcionalidades que **cierta** versión de un producto debe cumplir. Estos documentos han sido adaptados y llevados a lo que se conoce como AI-driven PRD o Prompt Requirement Document, que es un documento optimizado para la generación/implementación utilizando Inteligencia Artificial.

Pautas para tener un buen AI-driven PRD

- El documento debe describir la versión en la que se estará trabajando y el nombre de la aplicación que se está trabajando.
- Debe tener un resumen del proyecto y arquitectura de las aplicaciones, *stacks* tecnológicos, contexto del proyecto, y datos de interés.
- Debe existir una definición de los Actores (Roles) que intervienen.
- Debe tener los requerimientos funcionales, separados por módulos, o unidades lógicas, con un identificador único para el requisito. Los requerimientos deben ser escritos en un lenguaje claro y poco rebuscado, describiendo funcionalidad, flujo y validaciones.
- Se deben definir requisitos no funcionales en caso de que existan.
- El Documento debe contener instrucciones para la AI sobre decisiones técnicas, elementos o reglas del negocio y detalles de arquitectura.

Output: Un documento de requerimientos detallado que sirve como la "fuente de verdad" para el agente.

Para el trabajo en la creación de documentos, mi sugerencia es usar modelos con buen razonamiento como GPT-5.4 o Claude Sonnet 4.6 o Gemini 3.1 Pro. Estos modelos tienen un buen balance entre razonamiento y precio y si ya usas Claude, es Claude Sonnet 4.6 una buena elección.

2. Configuración, Agent Skills, [CLAUDE.MD](#), [AGENTS.MD](#)

En este paso nuestro objetivo es optimizar como el modelo ve tu proyecto, las reglas que debe seguir, como seguirlas y que hacer desde el punto de vista de decisiones técnicas y de arquitectura.

Documentacion

CLAUDE.MD

La documentación principal de un proyecto que se trabaje con Claude debe ser el [CLAUDE.md]. En este documento deben estar de manera resumida las decisiones más importantes del proyecto, arquitectura, reglas, especificaciones técnicas y de tecnología.

[CLAUDE.md] es un archivo especial que Claude lee al inicio de cada conversación. Incluye comandos Bash, estilo de código y reglas de flujo de trabajo. Esto le da a Claude contexto persistente que no puede inferir sólo del código.

El comando /init analiza su código base para detectar sistemas de compilación, marcos de prueba y patrones de código, dándole una base sólida para refinar.

No hay un formato requerido para archivos [CLAUDE.md], pero manténlo corto y legible.

CLAUDE.md se carga cada sesión, así que solo incluya cosas que se apliquen ampliamente.

Para conocimiento de dominio o flujos de trabajo que sólo son relevantes a veces, use skills en su lugar. Claude los carga bajo demanda sin inflar cada conversación.

Manténgalo conciso. Para cada línea, pregúntese: “¿Causaría que Claude cometiera errores si elimino esto?” Si no, elimínalo. Los archivos [CLAUDE.md] inflados hacen que Claude ignore sus instrucciones reales.

Los archivos CLAUDE.md pueden importar archivos adicionales usando la sintaxis

@path/to/import, de esta manera se puede hacer una composición, para que [CLAUDE.md] se mantenga conciso y la documentación extra esté vinculada en el documento.

A menos que utilices herramientas con capacidades avanzadas (como *superpowers* instaladas) que implementen procesos de desarrollo guiados por pruebas, es crucial incluir una regla en tu archivo [CLAUDE.md] que le exija al modelo verificar la funcionalidad y asegurar que las pruebas pasen después de cada desarrollo.

En nuestro [CLAUDE.md] vamos a adicionar instrucciones adicionales para:

- Indicar dónde están los diseños de la aplicación y cómo leerlos.
- Dónde están los planes de desarrollo y cómo marcar como listas las tareas de estos planes.
- Utilize conventional commits.

Docs Folder

En la raíz de nuestro proyecto vamos a tener una carpeta donde pondremos toda la documentación relevante, los planes de desarrollo, los diseños, y la documentación en general de nuestro proyecto.

SKILLS

Los Skills son carpetas que contienen instrucciones, scripts y recursos que Claude descubre y carga dinámicamente cuando son relevantes para una tarea. Piense en ellas como manuales de capacitación especializados que le otorgan a Claude experiencia en dominios específicos.

Para poder instalar skills podemos hacerlo utilizando plugins de Claude o instalarlo utilizando CLI como el de <https://skills.sh/>

La idea con los skills es ir instalando los skills necesarios para el proyecto, y cuando surjan nuevos requerimientos que requieran otros skills adicionarlos.

Herramientas para la optimización de tokens

Muchas de estas herramientas están actualmente en desarrollo y cada día surgen nuevas, acá le dejo 2 que he probado y me han resultado

- Caveman: <https://github.com/juliusbrussee/caveman> Permite que la AI hable como un Caveman, lo que resulta en menor consumo de token.
- Rtk: <https://github.com/rtk-ai/rtk> Proxy para reducir token

Herramientas para la optimización de proceso de desarrollo

Estas herramientas en conjunto con los skills permiten que el proceso de desarrollo sea guiado por buenas prácticas y pautas aprobadas por equipos de desarrollo.

- Superpowers: <https://github.com/obra/superpowers>
- GetShitDone - <https://github.com/gsd-build/get-shit-done>
- Karpathy Skills - <https://github.com/forrestchang/andrei-karpathy-skills/issues>

Inicio proceso de Desarrollo

3. Creación de planes

Los planes son una parte fundamental en este proceso de desarrollo. Siempre lo más recomendable antes de empezar a programar es crear un plan de desarrollo que le permita a la AI investigar qué debe hacer y que, mediante la interacción con el operador (TÚ), pueda tomar decisiones para incluirlas en este plan. Para esto, Claude trae un modo que te permite cambiar a modo planificación.

La idea es que cuando ya tengas identificado en qué trabajar, se genera utilizando instrucciones como "Generar un plan de desarrollo para implementar la funcionalidad REF-RF-0001; el requisito está en el documento docs/prd-v0.1. Cada tarea debe estar de forma independiente y debe incluir un checkmark que debe ser marcado a medida que se vaya implementando."

Cada uno de estos planes generará, tareas que podrán ser marcadas como completadas utilizando [x] (Checkmark de Markdown).

4. Diseño Declarativo con Stitch/Claude Design u otra herramienta de diseño

En lugar de diseñar píxeles manualmente, se utiliza(<https://stitch.withgoogle.com/>) para generar UI mediante "Vibe Design" (describir la intención) o utilizando la nueva herramienta de claude design.

Estas 2 herramientas te permiten crear system designs y exportar las pantallas generadas como html, nuestro flujo de trabajo será iterar en la generación de la pantalla o en el cambio que queremos hacer según el requisito que estemos implementando.

Luego de tener las pantallas necesarias vamos a exportar a html en conjunto con una foto del diseño y lo pondremos dentro de docs/designs con un nombre sugerente a la funcionalidad o el id del requisito.

Nota: Si contamos ya con las pantallas diseñadas omitimos este paso y solo copiamos los diseños para docs/designs.

5. Proceso de desarrollo guiado por AI.

Gracias a la configuración, documentos de requisitos, diseños y planes que hemos desarrollado, nuestro proceso de desarrollo está listo para comenzar.

Para iniciar, le pediremos a Claude que ejecute el plan creado. Al hacerlo, el chat se pondrá en modo "Ask Permissions"; sin embargo, se recomienda cambiarlo a "Accept Edits" para un flujo de trabajo más ágil. Una vez finalizada la implementación, Claude se encargará automáticamente de realizar las pruebas, compilar y verificar que no existan problemas de tipos.

Una vez finalizado el proceso, disponemos de código listo para su revisión y prueba. Es sumamente importante examinar el código generado, verificar los archivos modificados y analizar sus implicaciones en otras funcionalidades. Si se detecta algún error o problema en la implementación, debe comunicarse a Claude con indicaciones precisas para su corrección. Si el problema resulta recurrente, se recomienda añadir una anotación en el archivo `CLAUDE.md` para evitar que se repita.

Al iterar y terminar nuestro proceso de desarrollo podemos pedir a claude que haga el commit por nosotros o nosotros mismos hacerlo y finalizar el proceso creando un Pull Request.

Diagrama del flujo de trabajo

https://drive.google.com/file/d/1UCDGJH0c_LQaoXL8kAPLhagN3dXKs_bx/view?usp=sharing

Documentación

Best Practices Claude Code: <https://code.claude.com/docs/es/best-practices>

Claude Code Skills: <https://claude.com/blog/skills-explained>